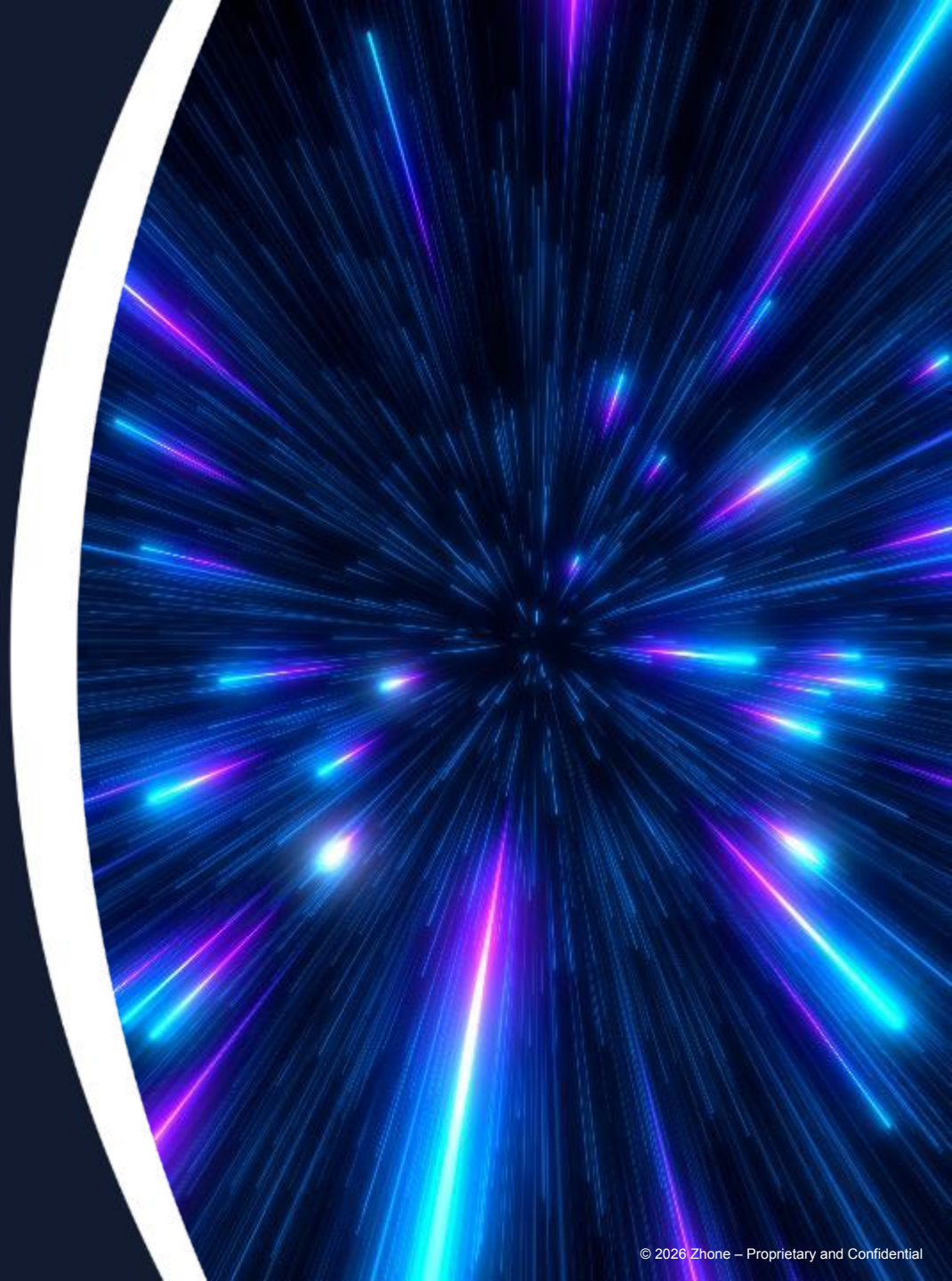
zhone

Daemon, IPC Overview: PIPE & FIFOs

Presenter: Nguyen Manh Hung
Date: 06/10/2026

Embedded Trainees



© 2026 Zhone – Proprietary and Confidential

OVERVIEW

1

Daemon

- Concept
- Characteristics
- Method of creation

2

IPC Overview

- List the types of IPC
- Compare IPCs

3

PIPE

- Concept
- PIPE size, block, SIGPIPE
- pipe and fork
- shell and pipe

4

FIFOs

- Concept
- create fifo
- multiple readers/writers

zhone
© 2026 Zhone – Proprietary and Confidential

01

Daemon

Long-lived processes running independently of terminal sessions.

1. Concept
2. Characteristics
3. Method of creation



zhone
© 2026 Zhone – Proprietary and Confidential

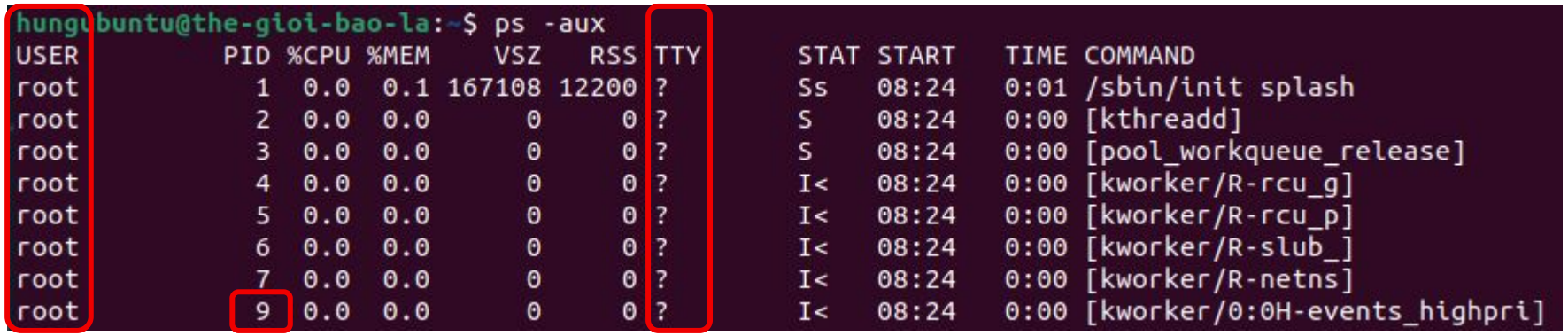
1. Daemon

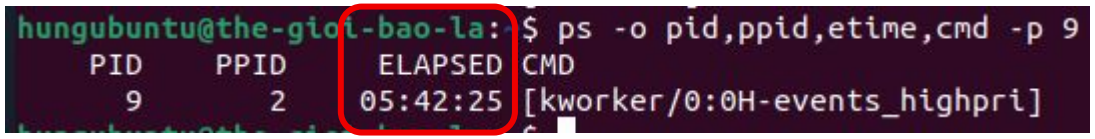
What is Daemon?

A daemon process is a background process that runs independently of any user control and performs specific tasks for the system.

Characteristics:

- Typically runs for extended periods.
- Terminal-independent.
- There is usually no direct interaction with the user.
- There is an IPC or network mechanism to provide the service.

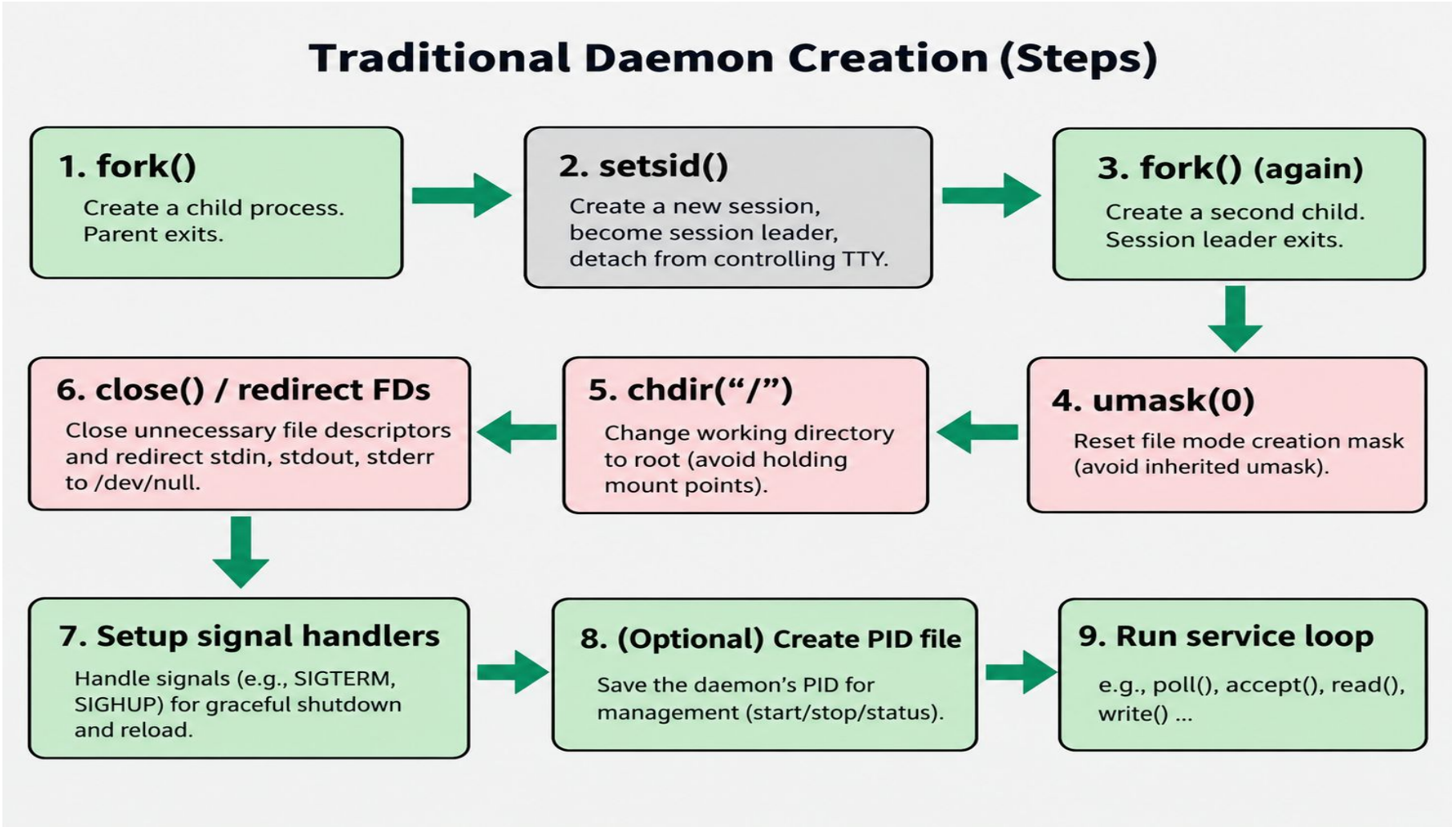




© 2026 Zhone – Proprietary and Confidential

1. Daemon

Traditional steps to create a daemon:



© 2026 Zhone – Proprietary and Confidential

1. Daemon

Traditional steps to create a daemon: Example code

```
1.  void init_daemon(void)
2.  {
3.      /* First fork: Detaches the child process from the parent
    */
4.      pid_t pid = fork();
5.      if (pid > 0)
6.      {
7.          /* parent exits */
8.          _exit(EXIT_SUCCESS);
9.      }
10.
11.     /* Create a new session and process group */
12.     setsid();
13.
14.     /* Second fork: daemon is no longer a session leader,
15.      * so it cannot acquire a controlling terminal. */
16.     pid = fork();
17.     if (pid > 0)
18.     {
19.         /* child 1 exits */
20.         _exit(EXIT_SUCCESS);
21.     }
```

```
1.      /* Redirect stdin/stdout/stderr */
2.      int fd = open("/dev/null", O_RDWR);
3.
4.      dup2(fd, STDIN_FILENO);
5.      dup2(fd, STDOUT_FILENO);
6.      dup2(fd, STDERR_FILENO);
7.
8.      /* Unmask file permissions */
9.      umask(0);
10.
11.     /* Change working directory to root */
12.     chdir("/");
13.
14.     /* Open System Logging */
15.     openlog("log_daemon", LOG_PID | LOG_NDELAY,
    LOG_DAEMON);
16.  }
```

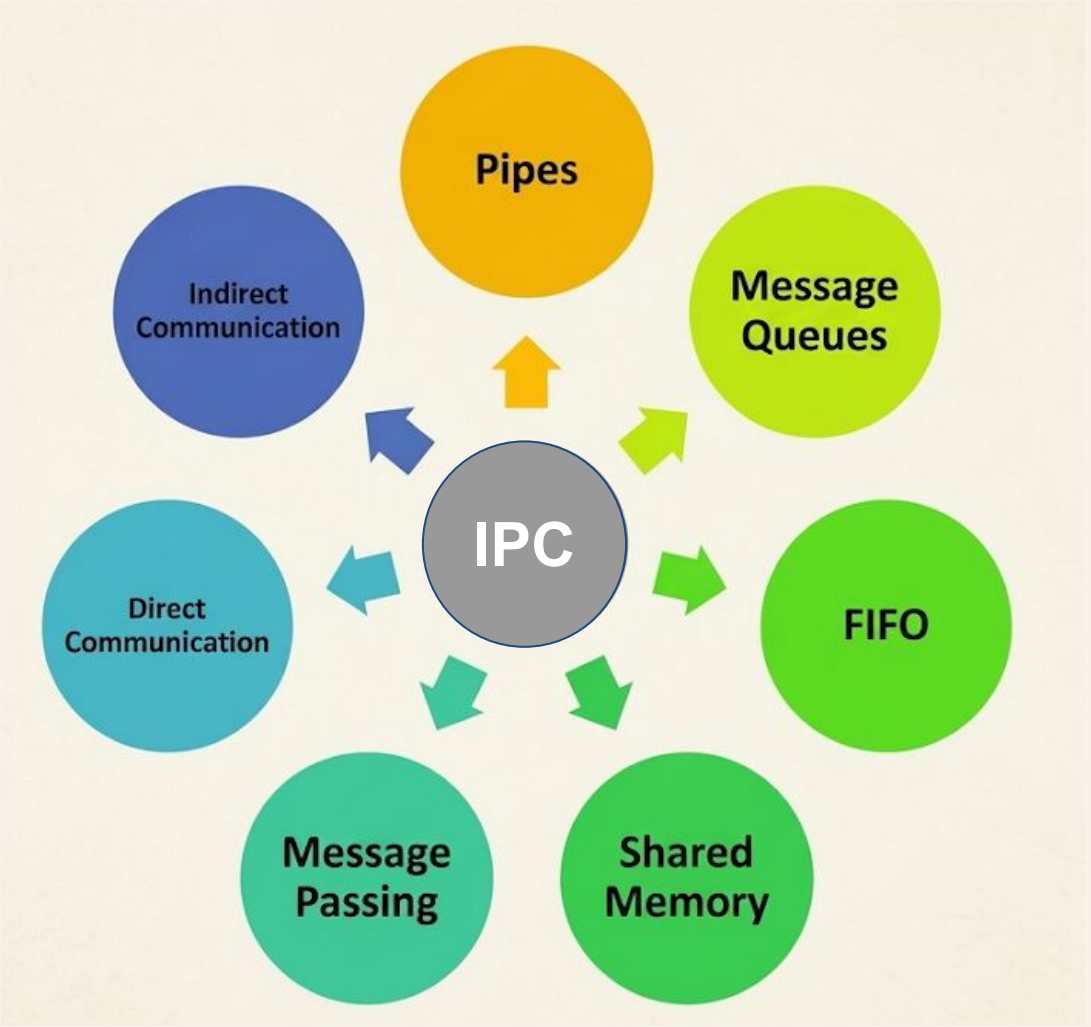
```
hungubuntu@the-gioi-bao-la:~$ ps -o pid,ppid,pgid,sid,tty,etime,cmd -p 53868
    PID    PPID    PGID     SID TT           ELAPSED CMD
  53868    2157   53867   53867 ?            00:33 ./daemon_process
hungubuntu@the-gioi-bao-la:~$
```


zhone
© 2026 Zhone – Proprietary and Confidential

02

IPC Overview

1. List the types of IPC
2. Compare IPCs



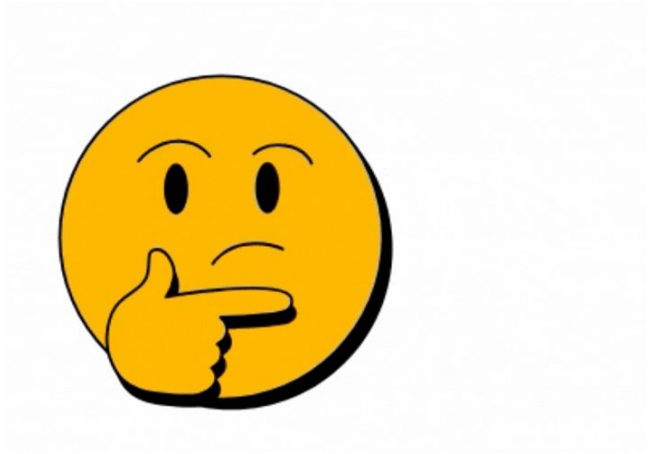
© 2026 Zhone – Proprietary and Confidential

2. IPC Overview

IPC (Inter-Process Communication) refers to mechanisms that allow processes to:

1. Exchange data (communication)
2. Synchronize activities (synchronization)
3. Notify of events (notification)

Why we need IPC?

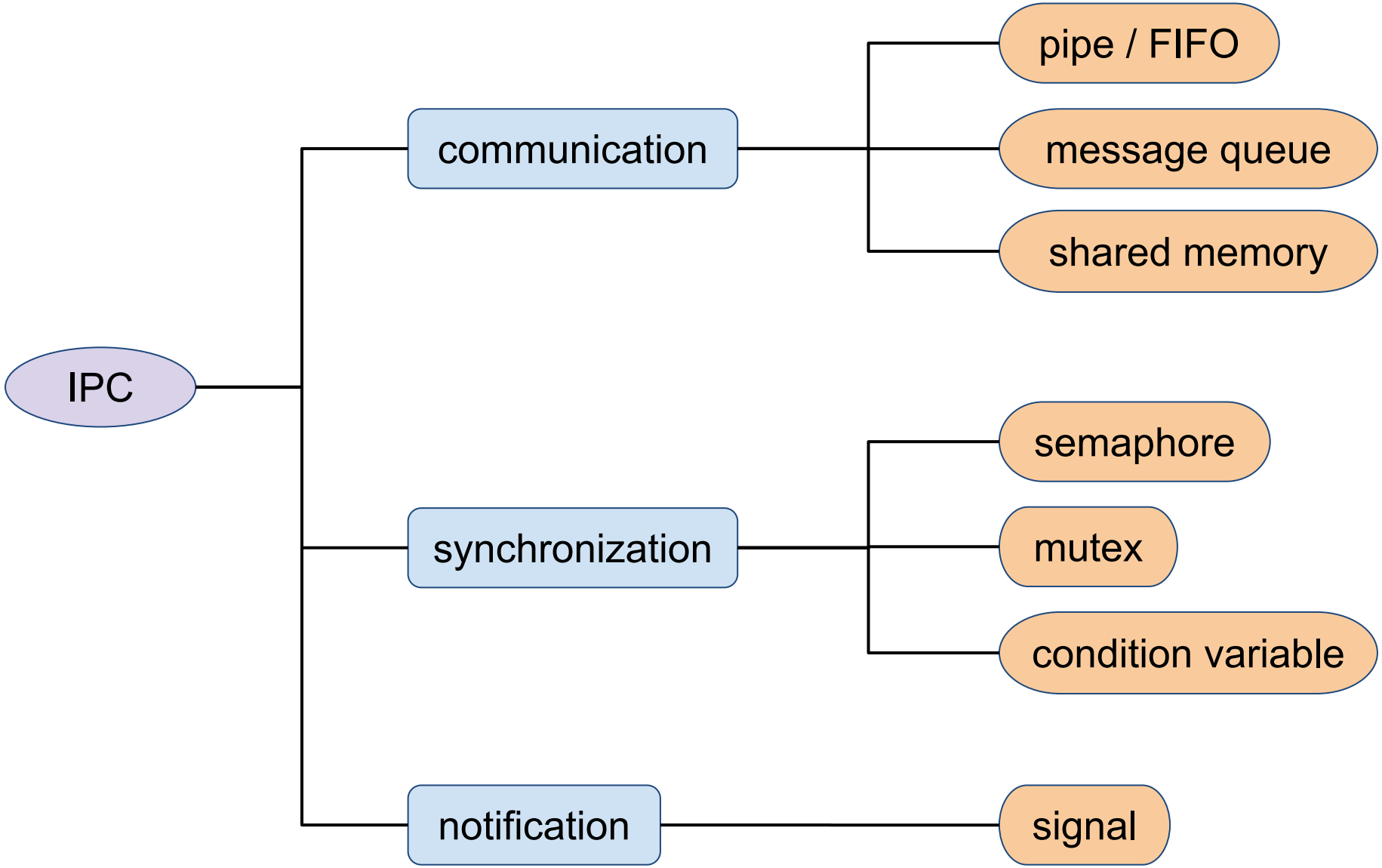


- **Information Sharing**: Multiple programs need access to identical data or shared files without duplicating them.
- **Computational Speedup**: Breaking a massive task into smaller subtasks running on separate processor cores requires fast data exchange.
- **Modularity**: Complex software uses separate, smaller modular processes that talk to each other cleanly rather than one giant monolithic block.

zhone
© 2026 Zhone – Proprietary and Confidential

2. IPC Overview

IPC tree:



zhone
© 2026 Zhone – Proprietary and Confidential

2. IPC Overview

Compare:

Mechanism	Main purpose	Data model	Typical use
Pipe	Data transfer	Byte stream	Parent ↔ child
FIFO	Data transfer	Byte stream	Unrelated processes
Message Queue	Data + command	Message	Command/event
Shared Memory	High-volume data, Fast transfer	Shared memory	Large Sensor / Image / Buffer
Semaphore	Synchronization	Counter	Producer / Consumer
Mutex	Mutual exclusion	Lock	Critical section
Condition Variable	Waiting/notification	Condition	Thread / Process Coordination
Signal	Notification	Event	Shutdown / Interrupt

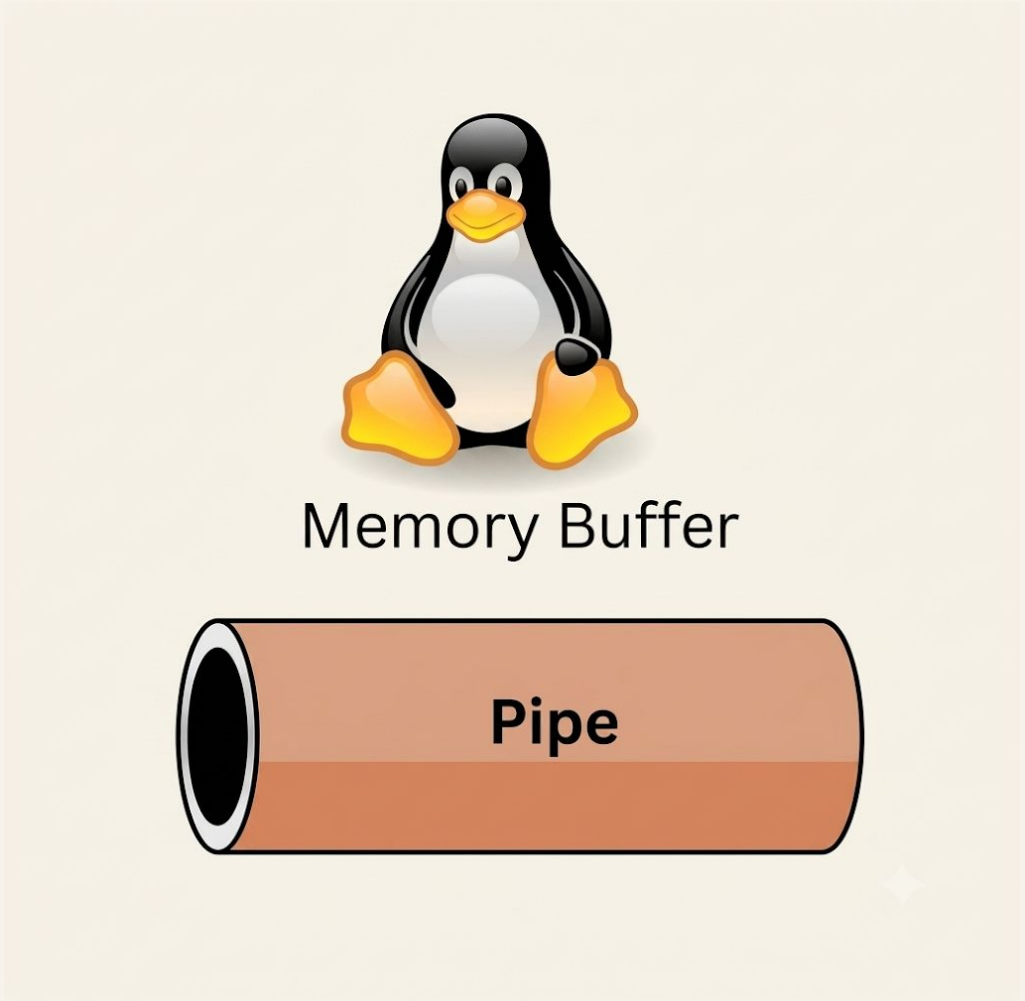
zhone
© 2026 Zhone – Proprietary and Confidential

03

PIPE

Unidirectional byte-stream interface.

1. Concept
2. PIPE size, block, SIGPIPE
3. pipe and fork
4. shell and pipe



zhone
© 2026 Zhone – Proprietary and Confidential

3. PIPE

What is Pipe?

A pipe is an IPC (Inter-Process Communication) mechanism that allows one process to transmit a byte stream to another process via the kernel.

Characteristics:

- byte stream
- unidirectional
- kernel-managed buffer
- highly suitable for communication between related processes, such as parent and child

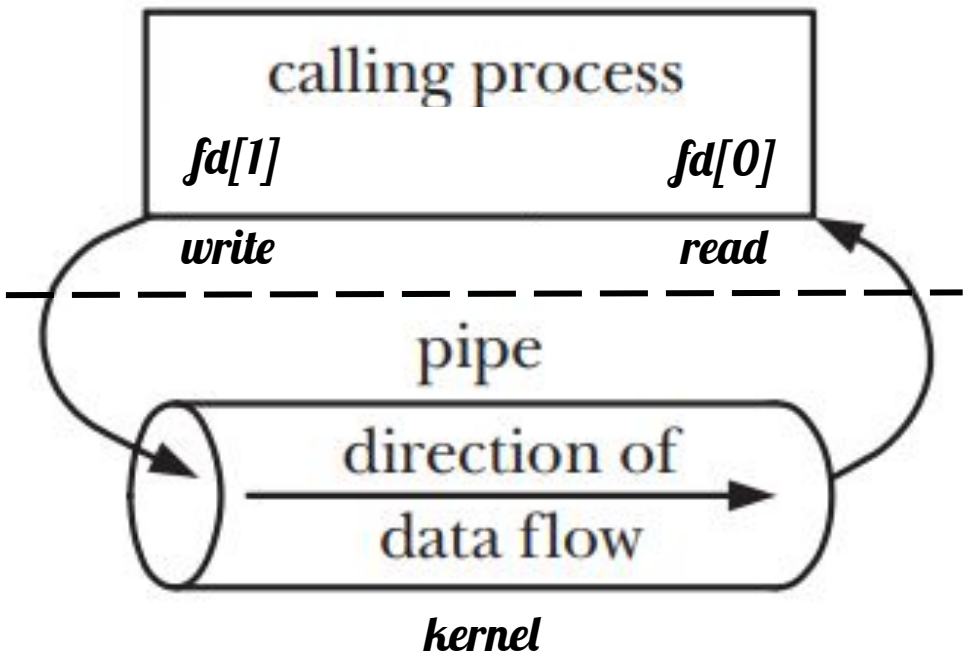
Suitable for	Not suitable for
Data streaming	Complex message structures
Parent–child communication	Random access
Simple producer–consumer scenarios	Large shared datasets
Passing stdin / stdout	Multiple clients requiring flexible communication

© 2026 Zhone – Proprietary and Confidential

3. PIPE

Some API of Pipe:

API	Role in Pipe Communication	Return
`pipe(int fd[2])`	Creates a pipe object: `fd[0]` → read end `fd[1]` → write end	`0` success `-1` failure
`write(fd, buf, size)`	Sends bytes into the pipe buffer	Bytes written `-1` on error
`read(fd, buf, size)`	Receives bytes from the pipe buffer	Bytes read `0` `-1` on error
`close(fd)`	Releases unused pipe ends	`0` success `-1` failure



zhone
© 2026 Zhone – Proprietary and Confidential

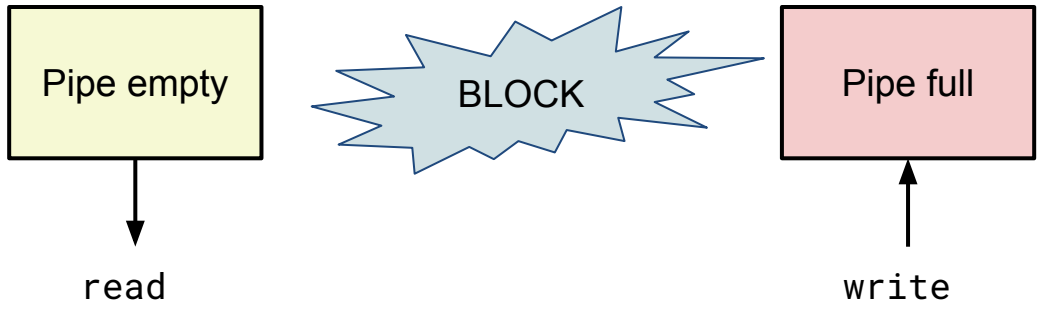
3. PIPE

Limit of pipe (defaut is 65536 byte (64kB))

```
1.  int main(int argc, char *argv[])
2.  {
3.      int n = 0;
4.      int fd[2];
5.      pipe(fd);
6.
7.      const char *msg = "H";
8.      while (1)
9.      {
10.         write(fd[1], msg, strlen(msg));
11.         n++;
12.         printf("write %d time\n", n);
13.     }
14.     return 0;
15. }
```

```
write 65531 time
write 65532 time
write 65533 time
write 65534 time
write 65535 time
write 65536 time
```

Blocking in pipe



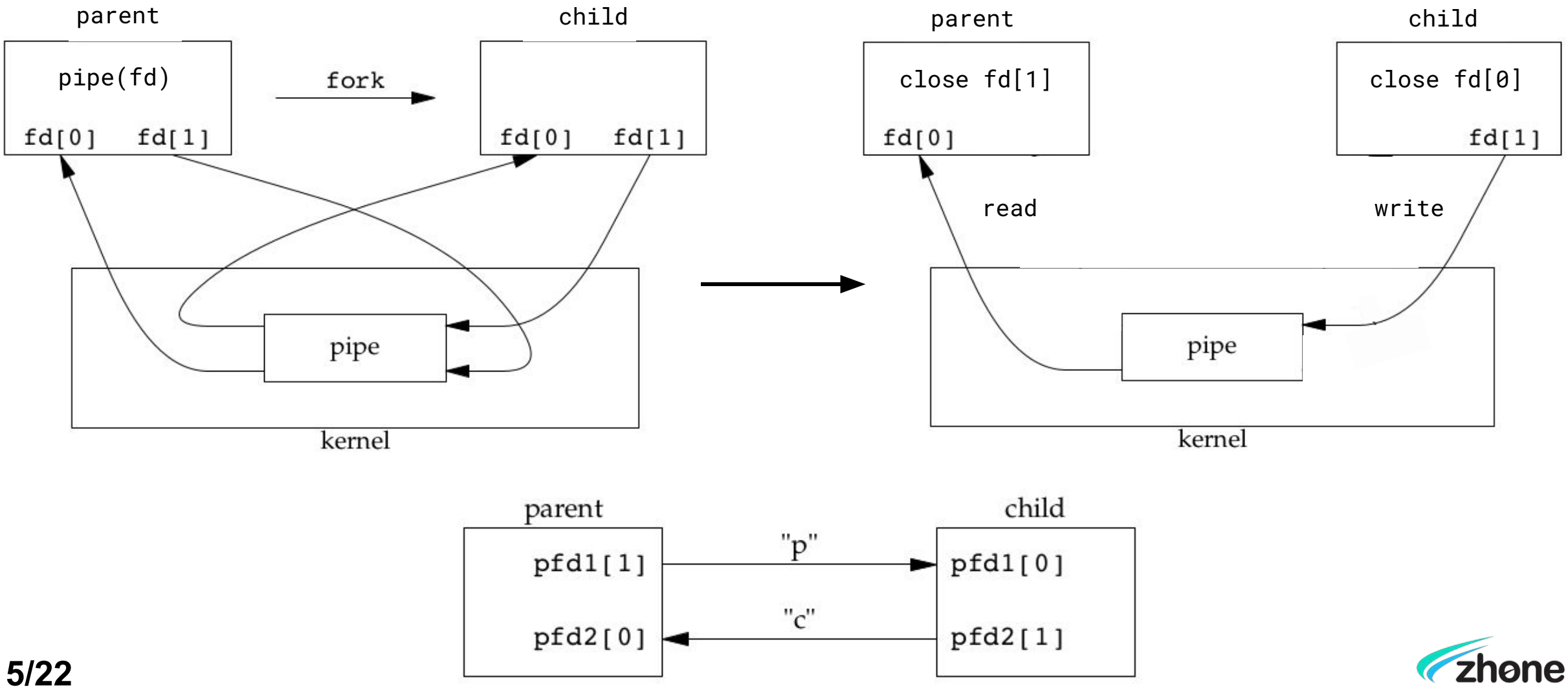
SIGPIPE

```
1.      int fd[2];
2.      pipe(fd);
3.
4.      close(fd[0]);
5.
6.      const char *msg = "HELLO";
7.      int ret = write(fd[1], msg, strlen(msg));
```


zhone
© 2026 Zhone – Proprietary and Confidential

3. PIPE

pipe + fork

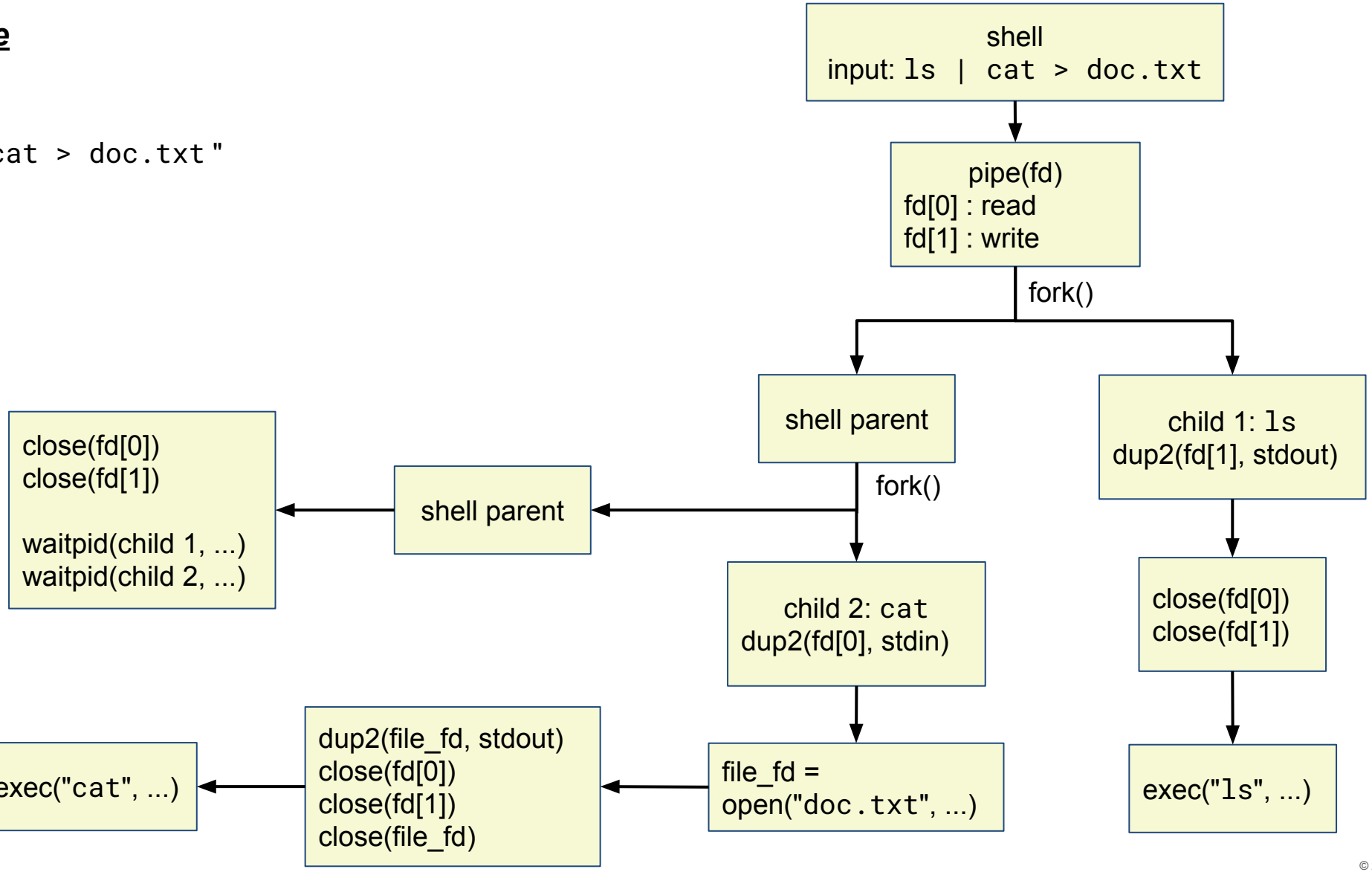


zhone
© 2026 Zhone – Proprietary and Confidential

3. PIPE

shell and pipe

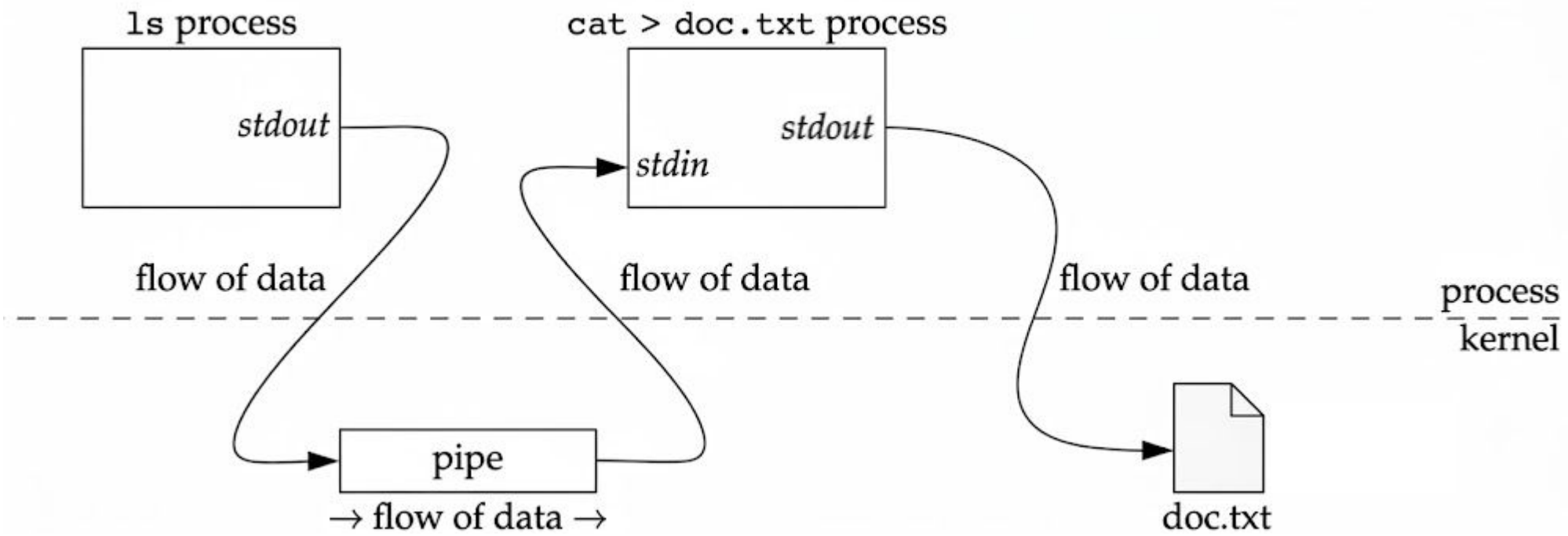
```
"ls | cat > doc.txt"
```



© 2026 Zhone – Proprietary and Confidential

3. PIPE

shell and pipe `"ls | cat > doc.txt"`



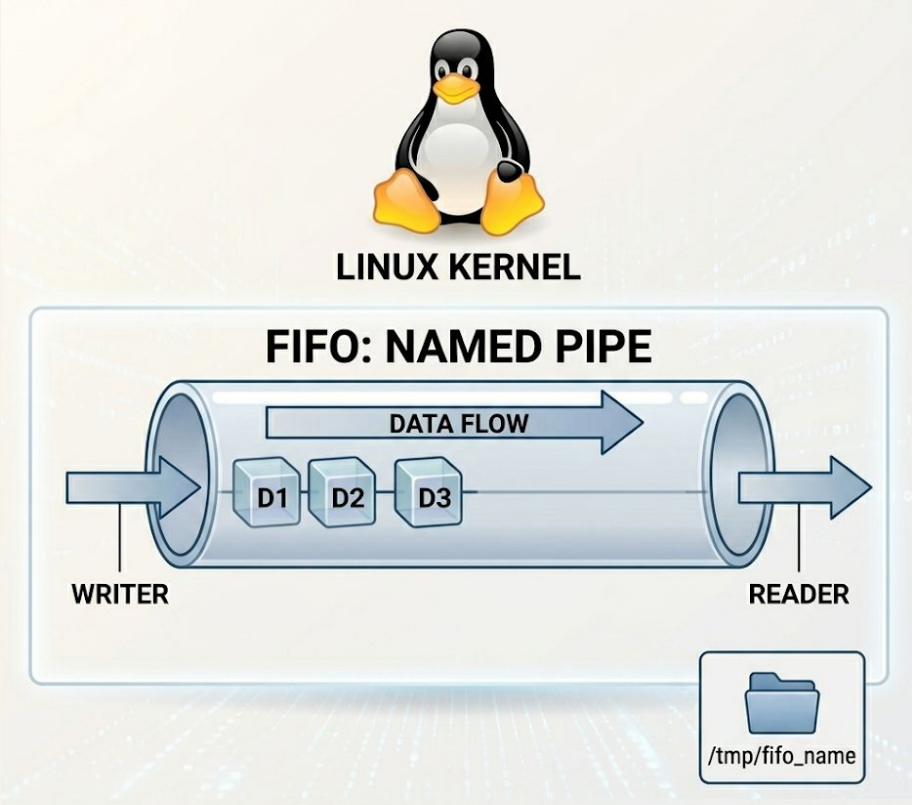
© 2026 Zhone – Proprietary and Confidential

04

FIFOs

Persistent communication paths.

1. Concept
2. Create fifo
3. Multiple readers/writers



zhone
© 2026 Zhone – Proprietary and Confidential

4. FIFOs

What is FIFOs?
A FIFO (named pipe) is a pipe with a name in the filesystem, which lets unrelated processes communicate. A FIFO is created with mkfifo() and accessed with open() like an ordinary file.

```
mkfifo("/tmp/fifo_A_to_B", 0666);
mkfifo("/tmp/fifo_B_to_A", 0666);

/* open fifo */
int fd = open("/tmp/fifo_A_to_B", O_WRONLY);
int fd1 = open("/tmp/fifo_B_to_A", O_RDONLY);
```

Function	Role
`mkfifo()`	Create an IPC endpoint
`open()`	Connect the process to the FIFO
`read()`	Receive data
`write()`	Send data
`close()`	Close connect to the FIFO
`unlink()`	Mark to delete the FIFO name
`fcntl()`	On/Off *O_NONBLOCK*

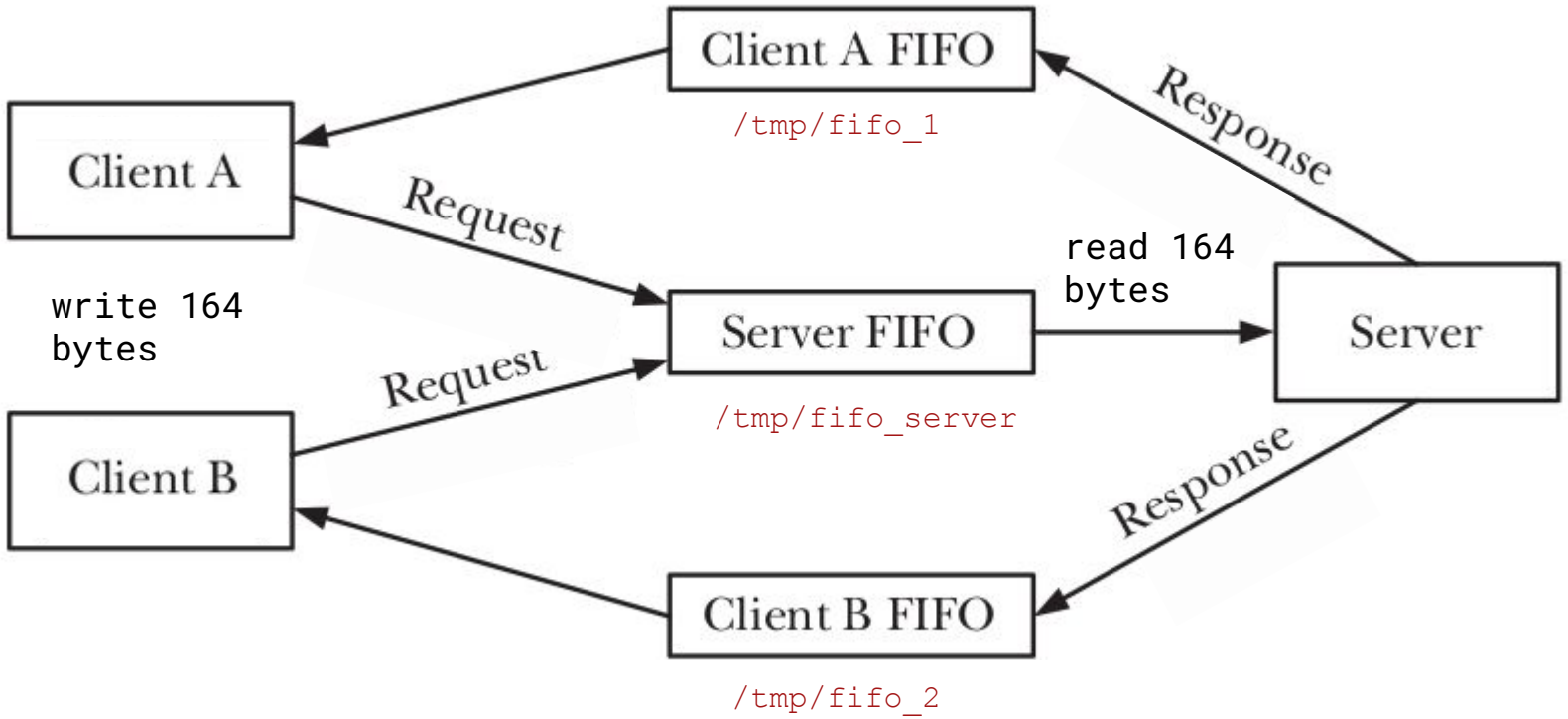
© 2026 Zhone – Proprietary and Confidential

4. FIFOs

A server - multiple client

```
struct new_client
{
    int id;
    char fifo_name[32];
    char data[128];
};
```

```
sizeof(struct new_client) =
164 bytes
```



zhone
© 2026 Zhone – Proprietary and Confidential

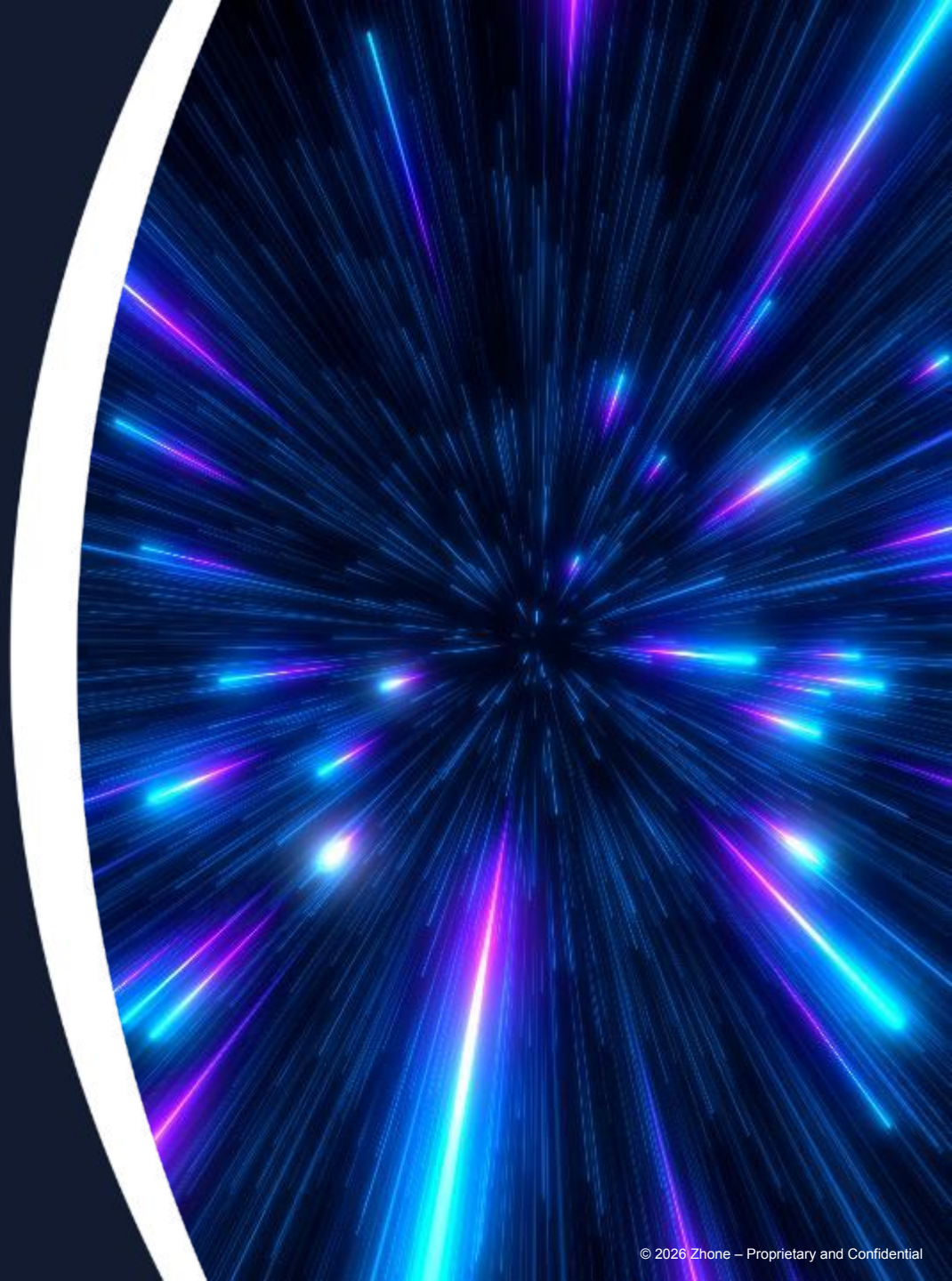
Synthetic

Pipe	**FIFO**
Anonymous	Named
pipe()	*mkfifo()*
File descriptors created directly	Pathname
Usually related processes	Can also be used by unrelated processes

© 2026 Zhone – Proprietary and Confidential

zhone

Thank you for listening...

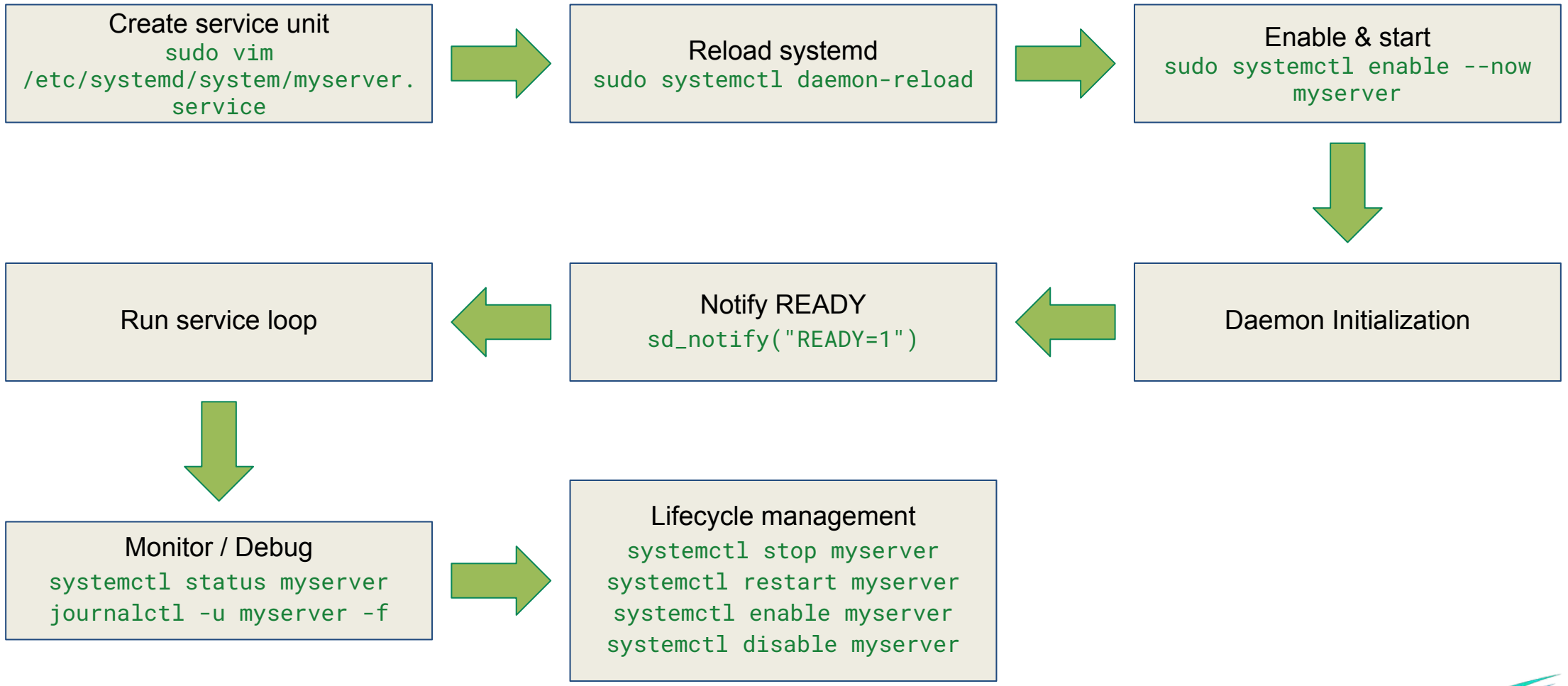


Embedded Trainees

© 2026 Zhone – Proprietary and Confidential

1. Daemon

Modern steps to create a daemon:

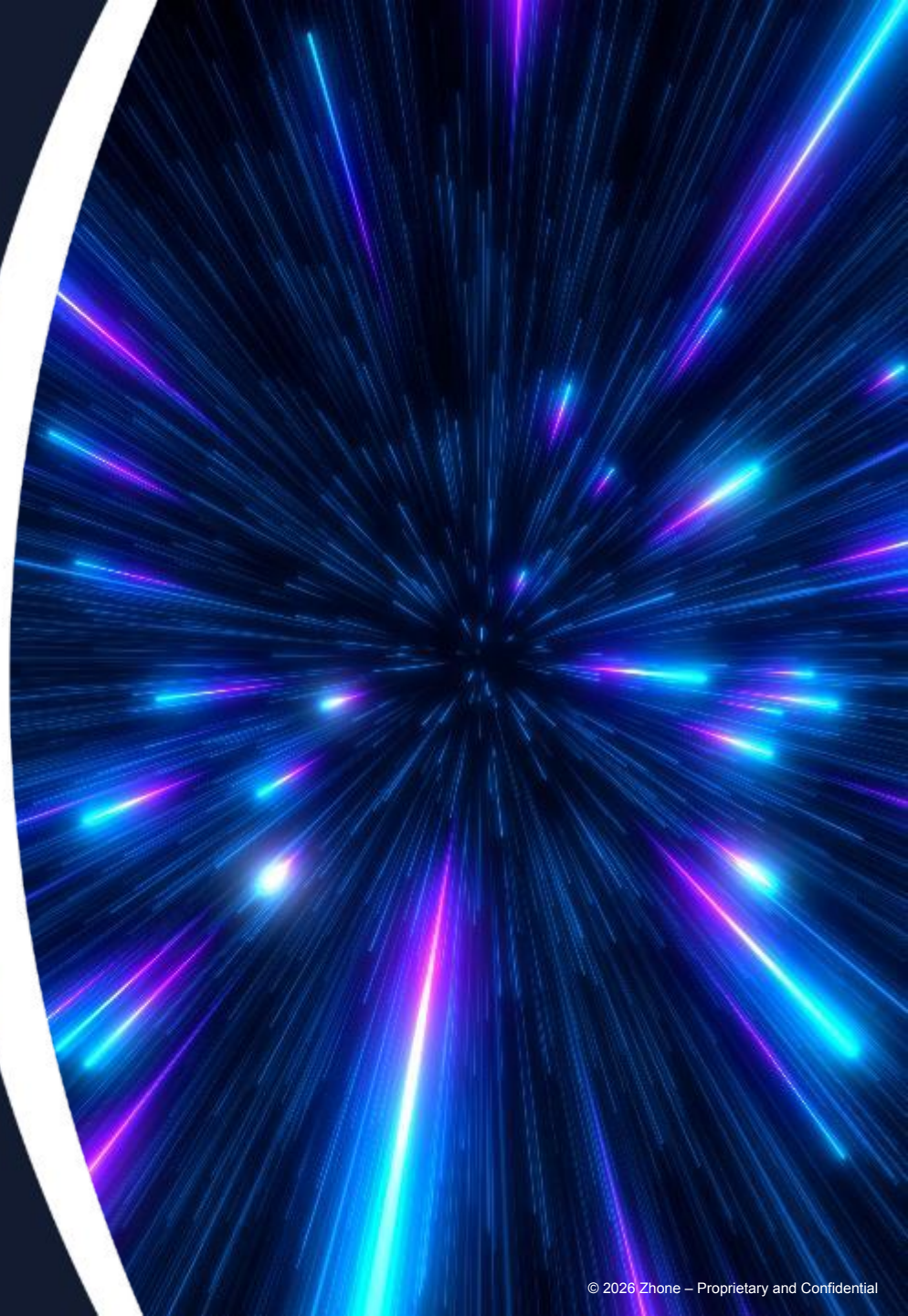


zhone
© 2026 Zhone – Proprietary and Confidential

zhone

Q&A

Embedded Trainees



© 2026 Zhone – Proprietary and Confidential

Q&A : 1. Compare Daemon and Background process

Criterion	Background Process	Daemon
Purpose	Run tasks without occupying the shell foreground	Provide long-running services independent of an interactive terminal
Launch method	./app &	systemctl start app
Terminal	May remain attached to the terminal	Typically independent of the terminal
stdin/stdout/stderr	May remain linked to the terminal	Usually redirected or managed via logging
Lifecycle management	Shell job control	Typically via systemd or supervisor
Auto-start on boot	Not by default	Configurable
Auto-restart on crash	Not by default	Configurable
Signal via PID	Yes (kill PID)	Yes (kill PID)
Example	sleep 300 &	sshd, cron

A daemon does not necessarily have to manually call `fork()` twice and `setsid()`. A service running directly under systemd can still adhere to the modern daemon model.

© 2026 Zhone – Proprietary and Confidential

Q&A : 2. Is redirecting to /dev/null mandatory or optional?

Redirecting *stdin*, *stdout*, and *stderr* to */dev/null* is optional, not mandatory, when creating a daemon. However, we need to handle these file descriptors in a manner appropriate for the program's purpose.

If not redirected to */dev/null*

File descriptor	If left unchanged	If redirected to /dev/null
stdin (FD 0)	May remain linked to the terminal or original input source	Reads typically return EOF
stdout (FD 1)	Output may still go to the terminal or original file	Written data is discarded
stderr (FD 2)	Error messages may still go to the terminal or original file	Error messages are discarded

If, after calling `setsid` and `close(STDIN_FILENO)`, we continue writing to `STDOUT_FILENO`, we may encounter a `SIGPIPE` signal that causes the program to terminate.

zhone

© 2026 Zhone – Proprietary and Confidential

Q&A :

3. After closing file descriptors (FDs), if I call `open`, will those FDs be reused?

After creating the daemon, you call `close(0)`, `close(1)`, and `close(2)`, then call `open()` to restore the terminal's original *stdin*, *stdout*, and *stderr*. **Will those fds be call back?**

The answer is: It cannot be restored automatically. `open()` does not know where the original file descriptors pointed, so it does not automatically reconnect to the old terminal.

© 2026 Zhone – Proprietary and Confidential

Q&A :

4. Why can the `setsid` function detach a daemon process from the terminal?

The process runs in the background but typically remains part of the shell session and retains a controlling terminal.

When the terminal closes, the following sequence of events may occur:

1. The terminal emulator closes the terminal device or PTY connection.
2. The kernel handles the terminal hangup.
3. The session leader or related processes may receive SIGHUP, depending on the specific mechanism and current state.
4. The shell may proceed to send SIGHUP to its jobs.
5. If the process receives SIGHUP and is using default behavior, it will terminate.

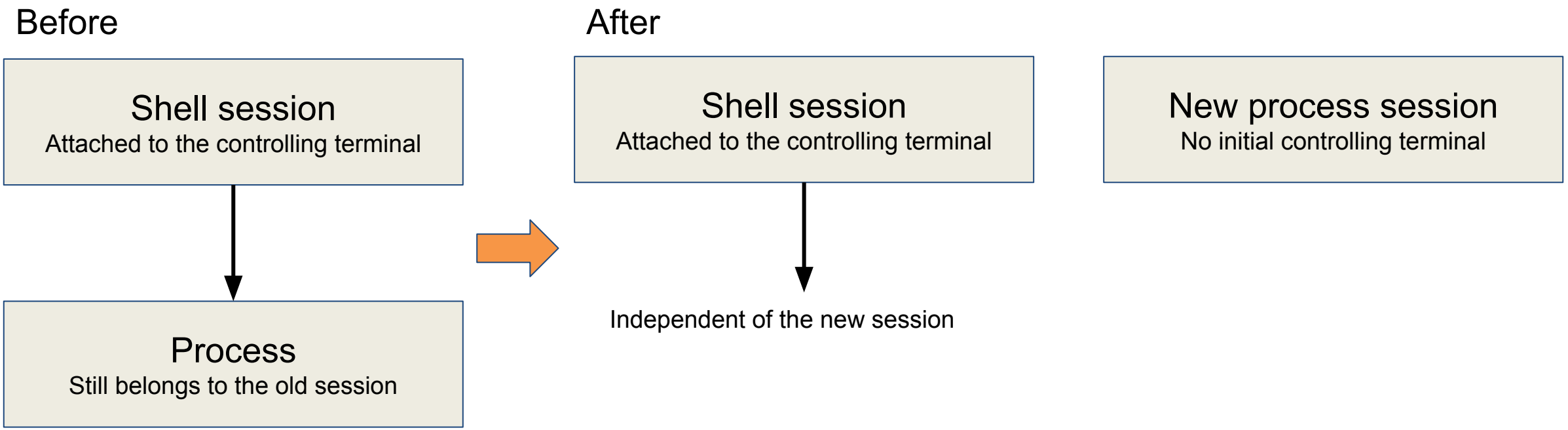
According to setsid(2), upon a successful call, the process:

- ➔ Becomes the **session leader** of the **new session**.
- ➔ Becomes the **process group leader** of the **new process group**.
- ➔ Has SID = PID and PGID = PID.
- ➔ Initially has **no controlling terminal**.

© 2026 Zhone – Proprietary and Confidential

Q&A :

4. Why can the `setsid` function detach a daemon process from the terminal?



When the old terminal closes, the hangup mechanism associated with the old controlling terminal no longer applies to the server in the same way. This is why `setsid()` is a crucial step in traditional daemonization.

© 2026 Zhone – Proprietary and Confidential

Q&A :

5. How is memory used when creating a pipe?

With a standard Linux pipe, the data flow proceeds through these steps:

1. Process A calls `write()`.
2. The kernel takes data from Process A's user-space memory and places it into the pipe buffer.
3. Process B calls `read()`.
4. The kernel transfers data from the pipe buffer to Process B's user-space memory.
5. The read data is removed from the pending data in the pipe.
6. Therefore, Process A and Process B do not need to share the same user-space memory.

How does the kernel organize pipe buffers?

At the Linux implementation level, a pipe has a management structure within the kernel, typically represented by the `*struct pipe_inode_info*`. This structure manages buffers using a ***ring buffer***.

A ring buffer typically has two positions:

- **Head** / read position: the position for the next data read.
- **Tail** / write position: the position for the next data write.

After each read or write operation, the corresponding position advances; upon reaching the end of the buffer, it wraps around to the beginning.

© 2026 Zhone – Proprietary and Confidential

Q&A :

6. Considerations when using pipes/FIFOs

Close file descriptors that are no longer in use:

- ➔ If a file descriptor (FD) for the writer remains open, the reader may not receive an EOF, even if the primary writing process has terminated. *-> Block forever.*
- ➔ If a file descriptor (FD) for the reader remains open, the writer may not receive a SIGPIPE, even if the intended receiving process has exited.
- ➔ Failure to properly close file descriptors (FDs) can lead to resource leaks.

FIFO: note that *open()* can block

- A FIFO differs from a regular pipe in one respect: *open()* may also wait for a process at the other end.
- In blocking mode, this command typically waits until a writer opens the FIFO. Similarly, a writer opening a FIFO in blocking mode may wait for a reader.
- Therefore, if an application consists of multiple services that start independently, it is necessary to plan the startup sequence, implement timeouts, or use an appropriate non-blocking opening method.

Pipes have a finite capacity.

- It is not possible to write an infinite amount of data into a pipe. When the buffer is full, a writer in blocking mode may be halted until the reader consumes the data.

zhone

© 2026 Zhone – Proprietary and Confidential